

FoFa — Technical Design Document

Audience: Whole cross-functional team (product, engineering, design, QA) **Last updated:** 2026-04-15

Table of Contents

1. [Product Overview](#)
 2. [Tech Stack](#)
 3. [Architecture](#)
 4. [Database Schema](#)
 5. [API Specification](#)
 6. [Frontend Component Structure](#)
 7. [Testing Strategy](#)
 8. [Accessibility Strategy](#)
 9. [Performance & Scalability](#)
 10. [Responsiveness Strategy](#)
 11. [Deployment](#)
-

1. Product Overview

FoFa (Foster Families) is a private community platform for foster families to connect, share updates, and coordinate playdates.

Problem

Foster families are often isolated. There is no dedicated space for them to share announcements, find other families nearby, communicate privately, or schedule child-friendly meetups. FoFa fills that gap.

Who It's For

Foster parents and their household members who want to build community with other local foster families.

Core Features

Feature	Description
Announcements Feed	Post text/media updates; comment and react with 5 emotion types
Direct Messaging	Private 1-on-1 conversations with unread count tracking
Family Profiles	Add/edit family member cards with photos and ages
Community Search	Find other verified foster families by name
Playdates	Post availability slots; send/accept/decline playdate requests
Auth	Email-verified registration, JWT sessions, password reset via email
Dark / Light Mode	User-toggled color scheme persisted to localStorage

What FoFa Does NOT Do

- Group messaging or channels
- Real-time push notifications (polling only)
- Public or anonymous access — every page requires a verified account
- Native mobile apps (the `mobile/` directory is a separate, out-of-scope concern)
- Payment processing or subscription management
- Content moderation tooling

Platform Scope

Web SPA (React) + REST API (Node.js). Mobile is listed in the monorepo but is outside this document's scope.

2. Tech Stack

Backend

Layer	Technology	Version	Purpose
Runtime	Node.js	LTS	JavaScript server runtime
Framework	Express	^4.18.3	HTTP routing and middleware
Language	TypeScript	^5.4.3	Static typing
Database	SQLite (better-sqlite3)	^12.8.0	Embedded relational DB, WAL mode
Auth tokens	jsonwebtoken	^9.0.2	JWT signing/verification (7-day expiry)
Password hashing	bcryptjs	^2.4.3	bcrypt with 12 rounds
Email	Nodemailer	^8.0.3	Gmail SMTP for verification + password reset
File uploads	Multer	^1.4.5-lts.1	Disk storage for thumbnails and media
Input validation	express-validator	^7.0.1	Request body/query validation
Security	Helmet	^7.1.0	HTTP security headers
Rate limiting	express-rate-limit	^7.2.0	200 req/15 min per IP
Compression	compression	^1.7.4	Gzip response compression
Logging	morgan	^1.10.0	HTTP request logging (dev mode)
Testing	Jest + ts-jest	^29.7.0 / ^29.1.2	Unit and integration tests

Frontend

Layer	Technology	Version	Purpose
UI library	React	^18.2.0	Component-based UI
Language	TypeScript	^5.4.3	Static typing
Bundler	Vite	^5.2.6	Dev server and production builds

Layer	Technology	Version	Purpose
Routing	React Router v6	^6.22.3	Client-side navigation
Server state	TanStack Query	^5.28.4	Data fetching, caching, and background refetch
Client state	Zustand	^4.5.2	Auth and theme stores (persisted to localStorage)
HTTP client	Axios	^1.6.8	API calls with JWT interceptor
Forms	React Hook Form	^7.51.1	Controlled forms without re-render overhead
Styling	Tailwind CSS	^3.4.19	Utility-first CSS
Toasts	react-hot-toast	^2.4.1	Non-blocking user feedback
Date utils	date-fns	^3.6.0	Date formatting and arithmetic
File drop	react-dropzone	^14.2.3	Drag-and-drop file upload UI
Unit tests	Vitest + RTL	^1.4.0 / ^15.0.2	Component and hook tests
E2E tests	Cypress	^13.7.3	Full browser automation

Custom Theme Tokens

Token	Value	Tailwind Class
Brand green	#4d9463	brand / brand-dark / brand-light
Accent gold	#f0b24f	accent / accent-dark
Body font	Nunito	font-body
Heading font	Titan One	font-heading

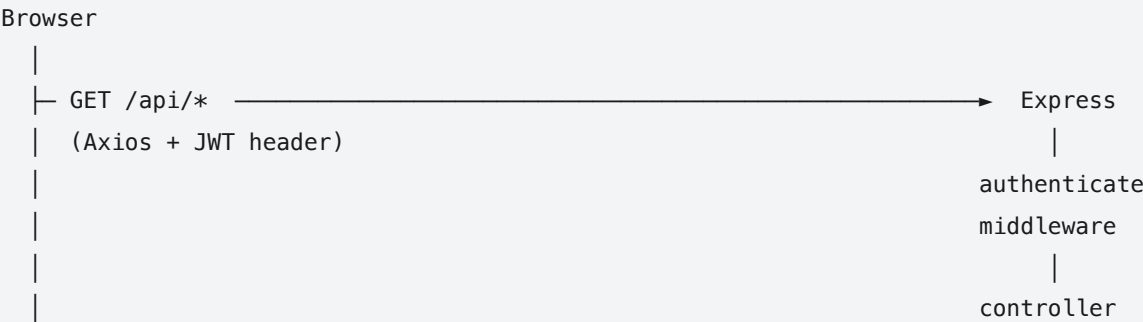
Dark mode is driven by a `dark` class on `<html>` and toggled via the `useThemeStore` Zustand store.

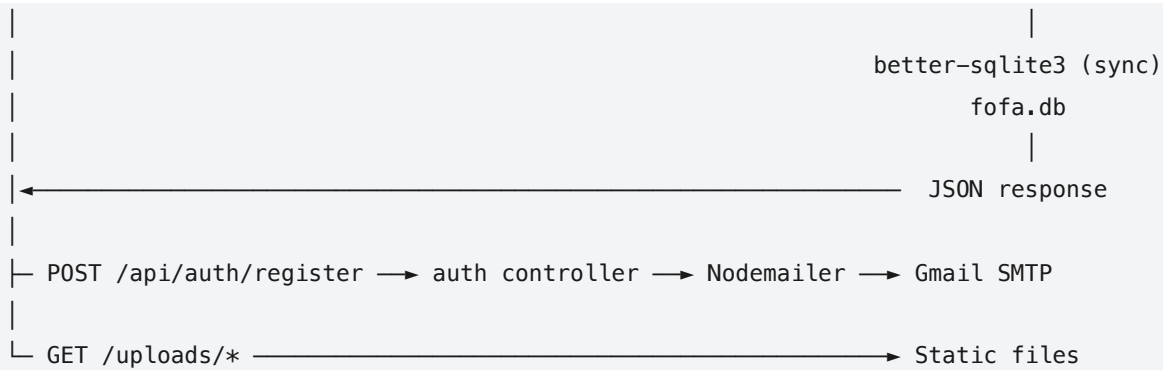
3. Architecture

Monorepo Structure

```
fofa/
├─ backend/                # Express REST API
│  └─ src/
│     ├─ controllers/      # Business logic per domain
│     ├─ routes/           # Route definitions (single index.ts)
│     ├─ middleware/       # auth, validate, upload
│     ├─ services/         # email.service.ts
│     └─ utils/
│        └─ db.ts          # SQLite singleton (WAL, FK ON)
│        └─ migrate.ts     # Schema migrations on startup
│  └─ tests/               # Jest test files
│  └─ uploads/             # Multer upload destination (gitignored)
│     └─ thumbnails/
│        └─ media/
├─ frontend/              # React SPA
│  └─ src/
│     ├─ components/       # Shared/reusable UI
│     ├─ contexts/         # Zustand stores (auth, theme)
│     ├─ hooks/            # Global reusable hooks
│     ├─ pages/            # Route-level components
│     ├─ services/         # Axios wrappers
│     └─ types/            # Shared TypeScript interfaces
│  └─ public/
├─ cypress/               # E2E specs
│  └─ e2e/                 # Test files (01-09)
│  └─ fixtures/
│  └─ support/
└─ mobile/                # React Native (out of scope)
```

Request Flow





Auth Architecture

1. **Register** — bcrypt-hashed password stored; UUID email token created and emailed (24 h TTL).
2. **Verify email** — token checked for `used = 0` and `expires_at` ; user marked `verified = 1` .
3. **Login** — credentials verified; JWT signed with `JWT_SECRET` (7-day expiry) returned to client.
4. **Authenticated requests** — `authenticate` middleware extracts `userId` from `Authorization: Bearer <token>` header, attaches it to `req.userId` .
5. **Password reset** — similar token flow; account enumeration prevented (identical response whether email exists or not).
6. **Token storage** — JWT stored in Zustand `fofa-auth` key in `localStorage` ; auto-injected by Axios request interceptor. On 401 (non-auth endpoint), Axios response interceptor calls `logout()` and redirects to `/login` .

4. Database Schema

ER Overview

```

users —< email_tokens
      —< password_reset_tokens
      —< family_members
      —< announcements —< comments
                        —< reactions
      —< messages (sender / receiver)
      —< availability_slots —< playdate_requests (requester / owner)
  
```

All primary keys are UUIDs (TEXT). Timestamps are stored as ISO-8601 TEXT (SQLite `datetime('now')`).

users

Column	Type	Constraints
id	TEXT	PRIMARY KEY
email	TEXT	UNIQUE NOT NULL
password	TEXT	NOT NULL (bcrypt hash)
name	TEXT	NOT NULL
city	TEXT	NOT NULL
state	TEXT	NOT NULL
thumbnail	TEXT	nullable (relative path under <code>/uploads</code>)
verified	INTEGER	NOT NULL DEFAULT 0 (0 = unverified, 1 = verified)
created_at	TEXT	NOT NULL DEFAULT <code>datetime('now')</code>
updated_at	TEXT	NOT NULL DEFAULT <code>datetime('now')</code>

email_tokens

Column	Type	Constraints
id	TEXT	PRIMARY KEY
user_id	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
token	TEXT	UNIQUE NOT NULL
expires_at	TEXT	NOT NULL
used	INTEGER	NOT NULL DEFAULT 0
created_at	TEXT	NOT NULL DEFAULT <code>datetime('now')</code>

password_reset_tokens

Column	Type	Constraints
<code>id</code>	TEXT	PRIMARY KEY
<code>user_id</code>	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
<code>token</code>	TEXT	UNIQUE NOT NULL
<code>expires_at</code>	TEXT	NOT NULL
<code>used</code>	INTEGER	NOT NULL DEFAULT 0
<code>created_at</code>	TEXT	NOT NULL DEFAULT <code>datetime('now')</code>

family_members

Column	Type	Constraints
<code>id</code>	TEXT	PRIMARY KEY
<code>user_id</code>	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
<code>name</code>	TEXT	NOT NULL
<code>age</code>	INTEGER	NOT NULL (0–120)
<code>thumbnail</code>	TEXT	nullable
<code>created_at</code>	TEXT	NOT NULL DEFAULT <code>datetime('now')</code>
<code>updated_at</code>	TEXT	NOT NULL DEFAULT <code>datetime('now')</code>

announcements

Column	Type	Constraints
<code>id</code>	TEXT	PRIMARY KEY

Column	Type	Constraints
<code>user_id</code>	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
<code>content</code>	TEXT	NOT NULL
<code>media_url</code>	TEXT	nullable
<code>media_type</code>	TEXT	CHECK IN ('image', 'video', NULL)
<code>created_at</code>	TEXT	NOT NULL DEFAULT <code>datetime('now')</code>
<code>updated_at</code>	TEXT	NOT NULL DEFAULT <code>datetime('now')</code>

comments

Column	Type	Constraints
<code>id</code>	TEXT	PRIMARY KEY
<code>announcement_id</code>	TEXT	NOT NULL REFERENCES announcements(id) ON DELETE CASCADE
<code>user_id</code>	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
<code>content</code>	TEXT	NOT NULL
<code>created_at</code>	TEXT	NOT NULL DEFAULT <code>datetime('now')</code>
<code>updated_at</code>	TEXT	NOT NULL DEFAULT <code>datetime('now')</code>

reactions

Column	Type	Constraints
<code>id</code>	TEXT	PRIMARY KEY
<code>announcement_id</code>	TEXT	NOT NULL REFERENCES announcements(id) ON DELETE CASCADE
<code>user_id</code>	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE

Column	Type	Constraints
type	TEXT	NOT NULL CHECK IN ('like','love','hug','celebrate','support')
created_at	TEXT	NOT NULL DEFAULT <code>datetime('now')</code>
—	—	UNIQUE(announcement_id, user_id) — one reaction per user per post

messages

Column	Type	Constraints
id	TEXT	PRIMARY KEY
sender_id	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
receiver_id	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
content	TEXT	NOT NULL
read	INTEGER	NOT NULL DEFAULT 0
created_at	TEXT	NOT NULL DEFAULT <code>datetime('now')</code>

availability_slots

Column	Type	Constraints
id	TEXT	PRIMARY KEY
user_id	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
date	TEXT	NOT NULL (YYYY-MM-DD)
start_time	TEXT	NOT NULL (HH:MM)
end_time	TEXT	NOT NULL (HH:MM)
status	TEXT	NOT NULL CHECK IN ('free','busy') DEFAULT 'free'
note	TEXT	nullable

Column	Type	Constraints
created_at	TEXT	NOT NULL DEFAULT datetime('now')

playdate_requests

Column	Type	Constraints
id	TEXT	PRIMARY KEY
requester_id	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
owner_id	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
slot_id	TEXT	NOT NULL REFERENCES availability_slots(id) ON DELETE CASCADE
message	TEXT	nullable
status	TEXT	NOT NULL CHECK IN ('pending','accepted','declined') DEFAULT 'pending'
created_at	TEXT	NOT NULL DEFAULT datetime('now')
updated_at	TEXT	NOT NULL DEFAULT datetime('now')

Indexes Summary

Index Name	Table	Columns	Purpose
idx_announcements_user	announcements	user_id	Filter by author
idx_announcements_created	announcements	created_at DESC	Feed ordering
idx_comments_announcement	comments	announcement_id	Load comments per post
idx_reactions_announcement	reactions	announcement_id	Aggregate reactions per post
idx_messages_sender	messages	sender_id	Sent message lookup

Index Name	Table	Columns	Purpose
idx_messages_receiver	messages	receiver_id	Inbox lookup
idx_family_user	family_members	user_id	Load family per user
idx_slots_user_date	availability_slots	(user_id, date)	Calendar queries
idx_requests_requester	playdate_requests	requester_id	Sent requests
idx_requests_owner	playdate_requests	owner_id	Received requests

5. API Specification

Base URL: `http://localhost:4000/api` (dev) / configured via `VITE_API_URL` (prod)

Auth method: `Authorization: Bearer <JWT>` header on all protected routes.

File upload content type: `multipart/form-data` for any endpoint that accepts a `thumbnail` or `media` field.

Auth

Method	Path	Auth	Description
POST	<code>/auth/register</code>	No	Create account; sends verification email
GET	<code>/auth/verify-email?token=</code>	No	Mark email as verified (single-use token)
POST	<code>/auth/login</code>	No	Returns <code>{ token, user }</code> on success
POST	<code>/auth/forgot-password</code>	No	Sends reset email (enumeration-safe)
POST	<code>/auth/reset-password</code>	No	Resets password using emailed token
POST	<code>/auth/change-password</code>	Yes	Changes password for logged-in user

Users

Method	Path	Auth	Description
GET	/users/me	Yes	Returns the current user's profile
PUT	/users/me	Yes	Update profile; accepts multipart/form-data with optional thumbnail
GET	/users/search?q=	Yes	Search users by name (partial match)
GET	/users/:id	Yes	Get any user profile by ID

Family Members

Method	Path	Auth	Description
GET	/family	Yes	List caller's family members
POST	/family	Yes	Add a family member (multipart/form-data , requires name and age)
PUT	/family/:id	Yes	Update a family member (optional thumbnail)
DELETE	/family/:id	Yes	Remove a family member

Announcements

Method	Path	Auth	Description
GET	/announcements?page=	Yes	Paginated feed (newest first); responses include author, comment count, reactions
POST	/announcements	Yes	Create post (multipart/form-data , optional media file)
PUT	/announcements/:id	Yes	Update post content (own posts only)
DELETE	/announcements/:id	Yes	Delete post (own posts only)

Method	Path	Auth	Description
GET	/announcements/:id/comments	Yes	List comments on a post
POST	/announcements/:id/comments	Yes	Add a comment
DELETE	/announcements/:id/comments/:commentId	Yes	Delete a comment
POST	/announcements/:id/reactions	Yes	Toggle a reaction (creates if absent, deletes if present)

Reaction types: like , love , hug , celebrate , support . One reaction per user per announcement (UNIQUE constraint). Sending the same type again removes it.

Messages

Method	Path	Auth	Description
GET	/messages	Yes	List all conversations (latest message per partner)
GET	/messages/:partnerId?page=	Yes	Paginated message thread; marks messages as read automatically
POST	/messages	Yes	Send a message (receiverId , content)
GET	/messages/unread/count	Yes	Returns { count: number } of unread messages

Note: Fetching a thread (GET /messages/:partnerId) marks all messages in that thread as read = 1 server-side. The Navbar polls /messages/unread/count every 30 seconds.

Playdates

Method	Path	Auth	Description
GET	/playdates/availability/:userId	Yes	Get availability slots for any user
POST	/playdates/availability	Yes	Add an availability slot (date , start_time , end_time , optional status , note)

Method	Path	Auth	Description
PUT	/playdates/availability/:id	Yes	Update own slot
DELETE	/playdates/availability/:id	Yes	Delete own slot
GET	/playdates/requests	Yes	List all requests (sent and received)
POST	/playdates/requests	Yes	Request a playdate on a specific slot (slotId , optional message)
PUT	/playdates/requests/:id/respond	Yes	Accept or decline a request (status : accepted or declined)

Health

Method	Path	Auth	Description
GET	/health	No	Returns { status: "ok", env }

6. Frontend Component Structure

Routing Table

Route	Page	Auth Required
/login	LoginPage	No (guest only)
/register	RegisterPage	No (guest only)
/forgot-password	ForgotPasswordPage	No (guest only)
/reset-password	ResetPasswordPage	No (guest only)
/verify-email	VerifyEmailPage	No
/dashboard	DashboardPage	Yes
/family	FamilyPage	Yes

Route	Page	Auth Required
/messages	MessagesPage	Yes
/community	CommunityPage	Yes
/profile	ProfilePage	Yes
/members/:id	MemberProfilePage	Yes
/playdates	PlaydatesPage	Yes
/*	—	Redirects to /dashboard

Route protection is implemented with two layout wrappers: `ProtectedLayout` (redirects to `/login` if no token) and `GuestLayout` (redirects to `/dashboard` if already authenticated).

Pages

Page	Responsibility
LoginPage	Email/password login form
RegisterPage	Registration form (email, password, name, city, state)
ForgotPasswordPage	Request password reset email
ResetPasswordPage	Set new password via emailed token
VerifyEmailPage	Handles email verification link redirect
DashboardPage	Announcements feed with pagination, create post, reactions, comments
FamilyPage	Manage own family members (add, edit, delete)
MessagesPage	Direct message inbox and conversation thread
CommunityPage	Search for other users
ProfilePage	Edit own profile (name, city, state, avatar) and change password
MemberProfilePage	View another user's profile and family members
PlaydatesPage	Calendar view, manage availability slots, send/respond to requests

Shared Components

components/ui/

Component	Purpose
Avatar	User avatar with fallback initials
Button	Styled button with variants (primary, secondary, danger)
Input	Labeled input with error display
Modal	Accessible overlay dialog
Logo	FoFa brand logo SVG

components/dashboard/

Component	Purpose
Navbar	Top navigation bar + mobile bottom tab bar; shows unread message and playdate badges
CommentsSection	Expandable comments list and add-comment form

components/announcements/

Component	Purpose
AnnouncementCard	Single feed post with reactions, comment count, media
CreateAnnouncementForm	Rich post creation with optional media upload

components/family/

Component	Purpose
FamilyMemberCard	Displays a family member with edit/delete actions

components/messages/

Component	Purpose
<code>MessagesPanel</code>	Two-pane message UI: conversation list + thread

`components/playdates/`

Component	Purpose
<code>MonthCalendar</code>	Month-grid view for availability
<code>WeekCalendar</code>	Week-column view for availability slots
<code>TimePicker</code>	Time input control for slot start/end

`components/WithAuth`

Legacy HOC wrapper — superseded by `ProtectedLayout` in `App.tsx` but remains available.

Where to Put a New Component

If the component is used in only one feature: put it inside that feature's page folder (e.g., `pages/FamilyPage/`). If it is shared across two or more features: put it in `components/` under the appropriate subdirectory. Each component file should be PascalCase and contain exactly one exported component.

State Management Summary

State type	Tool	Store/location
Auth (user, JWT)	Zustand + persist	<code>src/contexts/authStore.ts</code> (key: <code>fofa-auth</code>)
Theme (dark/light)	Zustand + persist	<code>src/contexts/themeStore.ts</code>
Server data	TanStack Query	Inline <code>useQuery</code> / <code>useMutation</code> calls in <code>pages/components</code>
Local UI state	<code>useState</code>	Component-local

Server state is never duplicated into Zustand stores.

API Service Layer

All API calls go through `src/services/index.ts` , which re-exports domain-grouped service objects. Each call is typed against the interfaces in `src/types/index.ts` .

Service	Methods
authService	register , login , verifyEmail , forgotPassword , resetPassword , changePassword
userService	getMe , updateProfile , search , getById
familyService	getAll , add , update , remove
announcementService	getAll , create , update , remove , getComments , addComment , deleteComment , toggleReaction
messageService	getConversations , getMessages , send , getUnreadCount
playdateService	getAvailability , addSlot , updateSlot , deleteSlot , getRequests , createRequest , respond

7. Testing Strategy

Three-Layer Table

Layer	Tool	Location	What it tests
Backend unit/integration	Jest + ts-jest	backend/tests/	Controllers, middleware, auth logic
Frontend unit/component	Vitest + React Testing Library	frontend/src/**/*.test.tsx and frontend/src/tests/unit/	Component rendering, user interactions, form validation
End-to-end	Cypress	cypress/e2e/	Full user journeys through the browser

Backend (Jest)

- Tests run in a Node environment against real SQLite (in-memory or temp DB recommended per test suite).
- Use `ts-jest` for TypeScript compilation.
- Coverage collected from `src/**/*.ts` excluding entry point and migrate utility.

Frontend (Vitest + RTL)

- Tests run in jsdom. Setup file mocks `import.meta.env` and `react-hot-toast`.
- Use `getByRole`, `getByLabelText`, `getByText` — never query by class or test ID unless unavoidable.
- Mock at the network boundary using module mocks for the service layer (MSW recommended for future adoption).
- No snapshot tests.

End-to-End (Cypress)

Specs cover the full user journey in numbered order:

Spec	Scenario
<code>01-registration.cy.ts</code>	Register new account
<code>02-login.cy.ts</code>	Login, invalid credentials
<code>03-announcements.cy.ts</code>	Create post, comment, react
<code>04-family.cy.ts</code>	Add/edit/delete family member
<code>05-messaging.cy.ts</code>	Send and receive messages
<code>06-profile.cy.ts</code>	Edit profile, change password
<code>07-community-navigation.cy.ts</code>	Search users, view profiles
<code>08-playdates.cy.ts</code>	Add slot, send/respond to request
<code>09-member-profile.cy.ts</code>	View another user's profile

TDD Workflow

1. Write a failing test that describes the expected behavior.

2. Run tests to confirm failure.
3. Implement the minimum code to make the test pass.
4. Refactor while keeping tests green.

Test Commands

```
# Backend unit tests
cd backend && npm test

# Backend tests in watch mode
cd backend && npm run test:watch

# Frontend unit tests (single run + coverage)
cd frontend && npm test

# Frontend tests in watch mode
cd frontend && npm run test:watch

# Cypress interactive runner
cd frontend && npm run cypress:open

# Cypress headless CI run
cd frontend && npm run cypress:run
```

Coverage Targets

Layer	Target
Backend statements	≥ 80%
Frontend statements	≥ 70%
E2E critical paths	100% of core flows covered

8. Accessibility Strategy

Target: WCAG 2.1 Level AA

FoFa is used by foster families who may include users with disabilities. Accessibility is a core quality bar, not a post-launch concern.

Principles

Principle	Implementation
Perceivable	All images have alt text or <code>aria-label</code> ; color is never the only differentiator
Operable	All interactive elements reachable and usable via keyboard
Understandable	Labels and instructions are clear; errors are field-level and descriptive
Robust	Semantic HTML; ARIA roles only where native semantics are insufficient

Implementation Checklist

Semantic HTML

- ☐ Use `<nav>` for navigation bars (Navbar already uses `aria-label="Main navigation"`)
- ☐ Use `<main>` , `<section>` , `<article>` for page structure
- ☐ Headings follow a logical hierarchy (h1 → h2 → h3)

Forms

- ☐ Every input has an associated `<label>` (via `for / id` or `aria-labelledby`)
- ☐ Validation errors rendered next to the field, not only in a toast
- ☐ Required fields indicated with `aria-required` or visible marker

Interactive Components

- ☐ Buttons use `<button>` (not `<div onClick>`)
- ☐ Modals trap focus and return focus on close
- ☐ Dropdown menus use `role="menu"` / `role="menuitem"` with keyboard navigation
- ☐ Active nav links set `aria-current="page"`
- ☐ Profile menu button sets `aria-expanded` and `aria-haspopup="menu"`
- ☐ Badge counts surfaced to screen readers via `aria-label` on nav links (e.g., "Messages, 3 pending")

Color & Contrast

- ☐ Brand green (#4d9463) on white background meets AA for large text; verify with a contrast checker for small text
- ☐ Dark mode tokens maintain equivalent contrast ratios
- ☐ Focus styles are visible (Tailwind `focus-visible:ring` or equivalent)

Testing Approach

Type	Tool
Automated lint	<code>eslint-plugin-jsx-a11y</code> (recommended addition)
Manual keyboard test	Tab through every interactive element on each page
Screen reader test	VoiceOver (macOS) on key flows (login, post creation, messaging)
Cypress a11y	<code>cypress-axe</code> (recommended addition)

9. Performance & Scalability

Frontend Performance

Concern	Strategy
Bundle size	Vite tree-shaking + code splitting per route (React lazy/Suspense — recommended)
API re-fetches	TanStack Query <code>staleTime: 30_000</code> default; feed uses manual pagination
Image loading	Thumbnails are small (5 MB upload limit, stored as-is — server-side resize recommended)
Unread polling	Navbar polls unread count and playdate requests every 30 s (not push-based)
Dark/light flash	Theme class applied before paint (Zustand persist reads synchronously from localStorage)

Backend Performance

Concern	Strategy
Response compression	<code>compression</code> middleware (gzip) on all responses
DB query performance	SQLite indexes on all foreign keys and common filter columns

Concern	Strategy
DB write throughput	WAL journal mode enables concurrent reads during writes
File I/O	Multier disk storage; heavy media uploads limited to 100 MB
Request abuse	Rate limit: 200 requests per 15 minutes per IP
Body size	JSON body limited to 10 MB

Scalability Constraints

Constraint	Threshold	Migration Path
SQLite concurrency	Single-writer; suitable for hundreds of concurrent users	Migrate to PostgreSQL when write contention emerges
Local file storage	Disk-bound; limited by server storage	Migrate uploads to S3-compatible object storage
In-process Node.js	Single process, no clustering	Add <code>cluster</code> module or move to containerized autoscaling
JWT stateless auth	No server-side token invalidation	Add a token revocation list (Redis) if forced logout is required

Caching Strategy

Layer	Current State	Recommended Next Step
API responses	None server-side	Add Cache-Control headers for public static assets
React Query	30 s stale time	Tune per-query based on volatility
Static assets	Vite content-hash filenames	CDN in front of the production build
DB query results	None	Add LRU cache for expensive read queries (e.g., community search)

10. Responsiveness Strategy

Breakpoints

Tailwind default breakpoints are used:

Breakpoint	Min Width	Target Devices
(default)	0 px	Mobile phones (portrait)
sm	640 px	Mobile landscape
md	768 px	Tablets, small laptops
lg	1024 px	Laptops
xl	1280 px	Desktops

The content max-width is capped at 1100 px (max-w-[1100px]).

Layout Strategy Per Section

Section	Mobile	Tablet (md)	Desktop
Navigation	Bottom tab bar (fixed)	Top navbar	Top navbar
Feed cards	Full-width single column	Full-width single column	Centered, max 1100 px
Messages	Full-screen single pane (conversation list or thread)	Two-pane side-by-side	Two-pane side-by-side
Family members	Single-column card list	Two-column grid	Three-column grid
Profile form	Stacked fields	Stacked fields	Constrained width centered
Playdates calendar	Month view, scrollable	Week view	Week view

Implementation Rules

- Use Tailwind responsive prefixes (md: , lg:) — never write custom @media queries.
- Mobile layout is the default (no prefix); larger-screen overrides are additive.

- Bottom tab bar shown on `< md` , hidden on `md:` and above (`md:hidden` / `hidden md:flex`).
- Profile dropdown and username are hidden on mobile nav to save space.
- Interactive tap targets meet the 44 × 44 px minimum on mobile.

Typography Scaling

Element	Mobile	Desktop
Body text	<code>text-sm</code> (14 px)	<code>text-base</code> (16 px)
Card title	<code>text-base</code>	<code>text-lg</code>
Page headings	<code>text-xl</code>	<code>text-2xl</code>
Nav labels	<code>text-[0.68rem]</code>	<code>text-[0.92rem]</code>

11. Deployment

Environment Variables

Backend (`backend/.env`)

Variable	Required	Description
<code>DATABASE_URL</code> / <code>DB_PATH</code>	No (default <code>./fofa.db</code>)	Path to SQLite database file
<code>JWT_SECRET</code>	Yes	Secret for signing JWTs (min 32 chars recommended)
<code>EMAIL_USER</code>	Yes	Gmail address for Nodemailer SMTP
<code>EMAIL_PASS</code>	Yes	Gmail app password for SMTP
<code>FRONTEND_URL</code>	Yes	Allowed CORS origin (e.g., <code>https://fofa.example.com</code>)
<code>PORT</code>	No (default <code>4000</code>)	Port the API server listens on

Variable	Required	Description
<code>UPLOADS_DIR</code>	No (default <code>./uploads</code>)	Directory for Multer file storage

Frontend (`frontend/.env`)

Variable	Required	Description
<code>VITE_API_URL</code>	No (default <code>http://localhost:4000/api</code>)	Base URL for API calls

Build & Start

```
# Backend – compile TypeScript and start
cd backend
npm run build      # tsc → dist/
npm start          # node dist/index.js

# Frontend – build static assets
cd frontend
npm run build      # tsc + vite → dist/
npm run preview    # preview the production build locally
```

Database Migrations

Migrations run automatically on every server start via `runMigrations()` in `src/utils/migrate.ts` . All `CREATE TABLE IF NOT EXISTS` and `CREATE INDEX IF NOT EXISTS` statements are idempotent — safe to run repeatedly.

To run migrations manually:

```
cd backend && npx ts-node src/utils/migrate.ts
```

File Uploads

Uploaded files are stored on the local filesystem at `backend/uploads/` . In a production deployment:

- Mount a persistent volume at the `UPLOADS_DIR` path, or

- Migrate to S3-compatible object storage and update the upload middleware and static file serving accordingly.

Recommended Production Stack

Concern	Recommendation
Process manager	PM2 (<code>pm2 start dist/index.js</code>) or a container (Docker)
Reverse proxy	Nginx — serves frontend static files and proxies <code>/api</code> and <code>/uploads</code> to Node.js
TLS	Let's Encrypt via Certbot on the Nginx layer
DB persistence	SQLite file on a mounted volume; daily backup via <code>sqlite3 .backup</code> or <code>litestream</code>
Frontend hosting	Nginx static file serving or CDN (Cloudflare Pages, Vercel)
Email	Gmail SMTP with an App Password, or migrate to SendGrid/SES for higher volume

CI/CD Checklist

- ☐ `cd backend && npm test` passes
- ☐ `cd frontend && npm test` passes
- ☐ `cd frontend && npm run build` exits 0 (TypeScript strict check passes)
- ☐ Environment variables injected via CI secrets (never committed)
- ☐ Database backed up before each production deploy
- ☐ Cypress smoke suite (`npm run cypress:run`) passes against staging before promoting to production